

21). //WAP to implement a class called circle that has private member variable for radius. Include member functions to calculate the circle's area and circumference. Also include another member function to display result.

```
#include<iostream>
#include<cmath>
using namespace std;

class Circle {
    private:
        float rad, area, circ;
    public:
        void setdata(float r){
            rad = r;
        }
        void Area(){
            area = M_PI *rad*rad;
        }
        void circumference(){
            circ = 2*M_PI*rad;
        }
        void display(){
            cout<<"The area of circle ="<<area<<endl;
            cout<<"The circumference of circle ="<<circ<<endl;
        }
};

int main () {
    int r;
    cout<<"Enter the radius of the circle: "<<endl;
    cin>>r;
    Circle r1;
    r1.setdata(r);
    r1.Area();
    r1.circumference();
    r1.display();
    return 0;
}
```

★ Output:

```
Enter the radius of the circle:
4
The area of circle =50.2655
The circumference of circle =25.1327

-----
Process exited after 7.405 seconds with return value 0
Press any key to continue . . .
```

22).

//WAP to create class called Person that has private member variables for name, age, and country. Implement member function to set and display values of these variables.

```
#include<iostream>
#include<string>
using namespace std;

class Person {
    private:
        string name;
        int age;
        string country;
    public:
        void setdata() {
            cout << "Enter the name of person: ";
            getline(cin, name);
            cout << "Enter the age: ";
            cin >> age;
            cin.ignore();
            cout << "Enter the person's country: ";
            getline(cin, country);
        }
        void Display() {
            cout << "\nPerson details:\n";
            cout << "Name   : " << name << endl;
            cout << "Age    : " << age << endl;
            cout << "Country : " << country << endl;
        }
};

int main() {
    Person d;
    d.setdata();
    d.Display();
    return 0;
}
```

★ Output:

```
Enter the name of person: Joshan Koirala
Enter the age: 20
Enter the person's country: Nepal

Person details:
Name      : Joshan Koirala
Age       : 20
Country   : Nepal

-----
Process exited after 21.71 seconds with return value 0
Press any key to continue . . .
```

23).

// Create a class called car that has data members company, model and year. Implement member functions to get and display values.

// Define member functions outside class.

```
#include<iostream>
using namespace std;

class Car{
    private:
        string company;
        int model;
        int year;
    public:
        void get ();
        void display ();
};

void Car::get(){
    cout<<"Enter the name of car company :";
    getline(cin, company);
    cout<<"Enter the car Model number : ";
    cin>>model;
    cout<<"Enter the manufacture year :";
    cin>>year;
}

void Car::display(){
    cout<<"\n Car details:\n";
    cout<<"Company name : "<<company<<endl;
    cout<<"Model      : "<<model<<endl;
    cout<<"Year       : "<<year<<endl;
}

int main() {
    Car c;
    c.get();
    c.display();
    return 0;
}
```

★ **Output:**

```
Enter the name of car company :BMW
Enter the car Model number : 750
Enter the manufacture year :2025
```

```
Car details:
Company name : BMW
Model       : 750
Year        : 2025
```

```
-----
Process exited after 11.53 seconds with return value 0
Press any key to continue . . .
```

24).

// WAP to create a class named BankAccount with private data member account_number and balance. Implement member functions to deposit and withdrawal money from the account. Also create member function to display balance.

// [Need to check whether there is sufficient balance or not while withdrawal].

```
#include <iostream>
using namespace std;

class BankAccount {
    private:
        long long int account_number;
        double balance;
    public:
        void setAccount(long long int no, double initial) {
            account_number = no;
            balance = initial;
        }

        void deposit(double amount) {
            if (amount > 0) {
                balance += amount;
                cout << "Deposited: " << amount << endl;
            }
            else {
                cout << "Invalid deposit amount!" << endl;
            }
        }

        void withdraw(double amount) {
            if (amount > balance) {
                cout << "Insufficient balance. Withdrawal failed!" << endl;
            }
            else if (amount <= 0) {
                cout << "Invalid withdrawal amount!" << endl;
            }
            else {
                balance -= amount;
                cout << "Withdrawn: " << amount << endl;
            }
        }

        void displayBalance() {
            cout << "Account Number: " << account_number << endl;
            cout << "Current Balance: " << balance << endl;
        }
};
```

```

    }
};

int main() {
    BankAccount acc;
    long long int acc_no;
    double initial, dep, wd;

    cout << "Enter account number: ";
    cin >> acc_no;
    cout << "Enter initial balance: ";
    cin >> initial;

    acc.setAccount(acc_no, initial);

    cout << "\n--- Deposit ---\nEnter amount to deposit: ";
    cin >> dep;
    acc.deposit(dep);

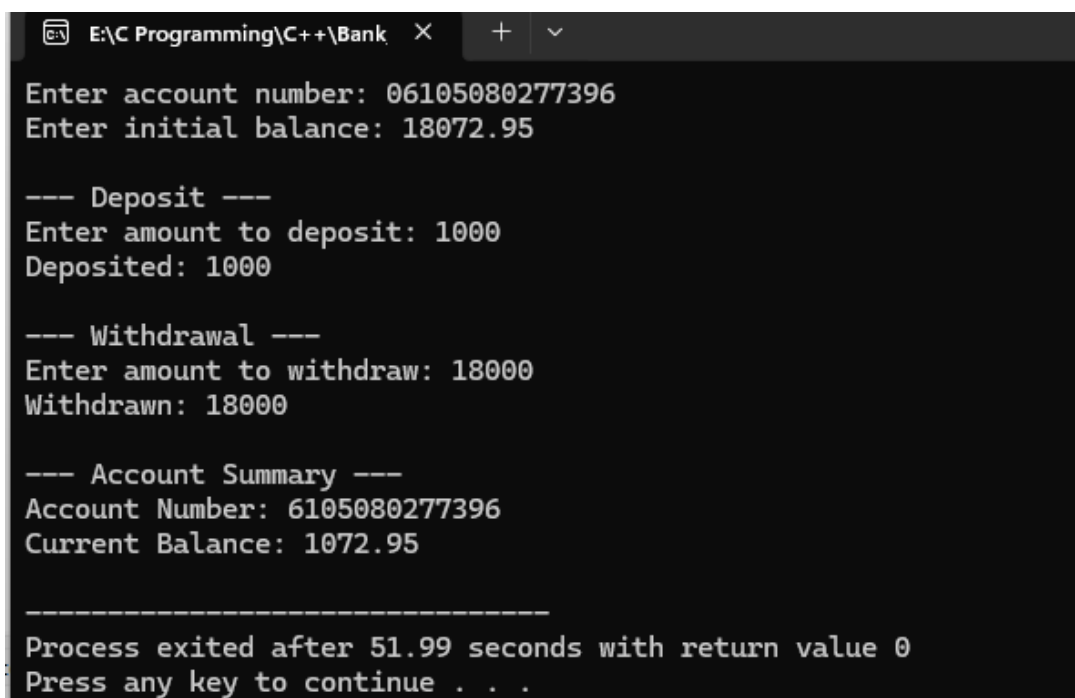
    cout << "\n--- Withdrawal ---\nEnter amount to withdraw: ";
    cin >> wd;
    acc.withdraw(wd);

    cout << "\n--- Account Summary ---" << endl;
    acc.displayBalance();

    return 0;
}

```

★ **Output:**



```

E:\C Programming\C++\Bank  X  +  v
Enter account number: 06105080277396
Enter initial balance: 18072.95

--- Deposit ---
Enter amount to deposit: 1000
Deposited: 1000

--- Withdrawal ---
Enter amount to withdraw: 18000
Withdrawn: 18000

--- Account Summary ---
Account Number: 6105080277396
Current Balance: 1072.95

-----
Process exited after 51.99 seconds with return value 0
Press any key to continue . . .

```

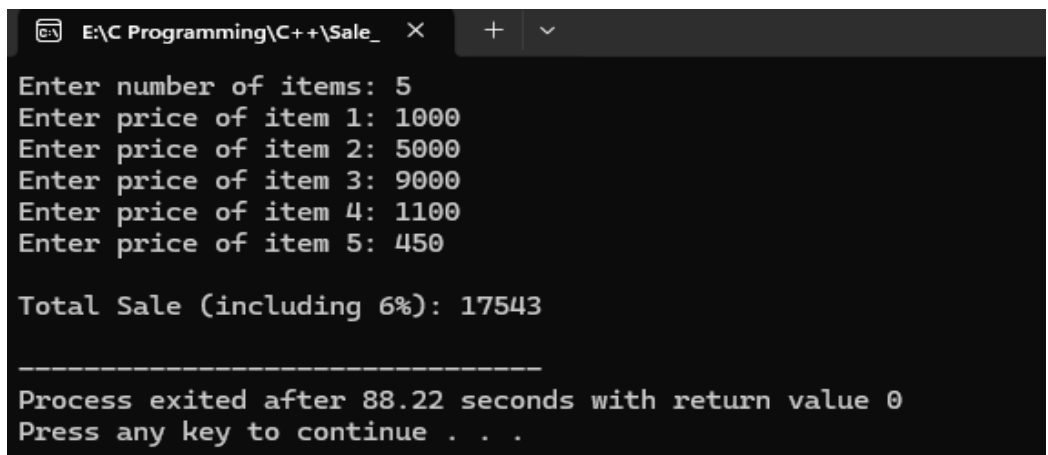
25).

// Demonstrate a class Sale that calculate the sale on an overall items, where sales is 6% on overall items. You must call a constructor that takes the sale rate as an argument and two-member function getTotal which returns total sale. Your program should print the TotalSale in main function.

```
#include <iostream>
using namespace std;
class Sale {
    private:
        float saleRate;
        float totalAmount;
    public:
        Sale(float rate) {
            saleRate = rate;
            totalAmount = 0;
        }
        void inputItems(int numItems) {
            float price;
            for (int i = 1; i <= numItems; i++) {
                cout << "Enter price of item " << i << ": ";
                cin >> price;
                totalAmount += price;
            }
        }
        float getTotal() {
            return totalAmount + (totalAmount * saleRate);
        }
};

int main() {
    int n;
    cout << "Enter number of items: ";
    cin >> n;
    Sale s(0.06);
    s.inputItems(n);
    float totalSale = s.getTotal();
    cout << "\nTotal Sale (including 6%): " << totalSale << endl;
    return 0;
}
```

★ Output:



The screenshot shows a Windows command prompt window titled "E:\C Programming\C++\Sale_". The program prompts the user to enter the number of items (5) and then the price of each item (1000, 5000, 9000, 1100, 450). It then displays the total sale including 6% tax as 17543. The window also shows the process exit message: "Process exited after 88.22 seconds with return value 0" and "Press any key to continue . . .".

```
E:\C Programming\C++\Sale_
Enter number of items: 5
Enter price of item 1: 1000
Enter price of item 2: 5000
Enter price of item 3: 9000
Enter price of item 4: 1100
Enter price of item 5: 450

Total Sale (including 6%): 17543

-----
Process exited after 88.22 seconds with return value 0
Press any key to continue . . .
```

26).

// Create a class 'Employee' having two private data members EmployeeName and EmployeeId.
 Create an object 'Employee1' of type Employee. Copy contents of Employee1 to Employee2.
 // [Both Shallow and Deep copy]

```
#include <iostream>
using namespace std;

class Employee {
    private:
        string EmployeeName;
        int EmployeeId;
    public:
        Employee() {
            EmployeeName = "";
            EmployeeId = 0;
        }
        Employee(string name, int id) {
            EmployeeName = name;
            EmployeeId = id;
        }
        // Deep Copy Constructor
        Employee(Employee& e) {
            EmployeeName = e.EmployeeName;
            EmployeeId = e.EmployeeId;
        }
        // Shallow copy
        void shallowCopy(Employee& e) {
            EmployeeName = e.EmployeeName;
            EmployeeId = e.EmployeeId;
        }
        void display() {
            cout << "Employee ID: " << EmployeeId << ", Name: " << EmployeeName << endl;
        }
};

int main() {
    Employee Employee1("Joshan", 9);

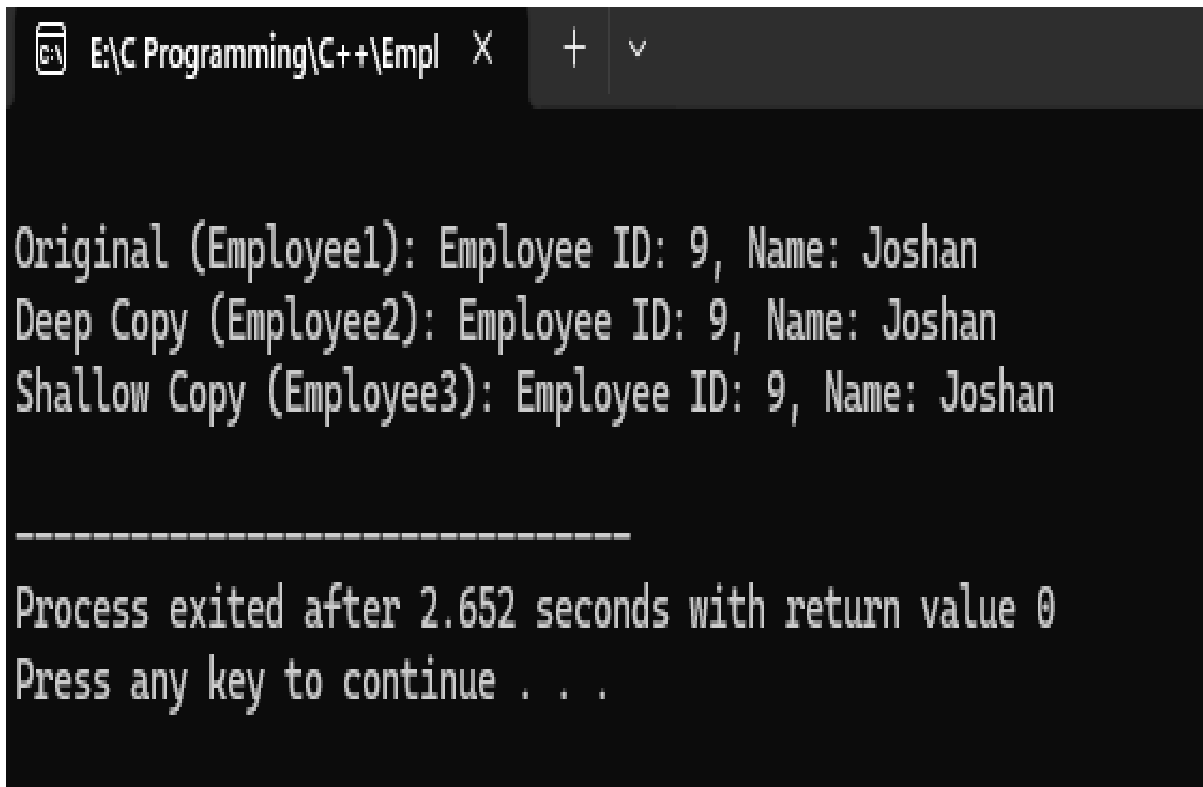
    // Deep Copy
    Employee Employee2 = Employee1;

    // Shallow Copy
    Employee Employee3;
    Employee3.shallowCopy(Employee1);

    cout << "\nOriginal (Employee1): ";
    Employee1.display();
```

```
    cout << "Deep Copy (Employee2): ";  
    Employee2.display();  
  
    cout << "Shallow Copy (Employee3): ";  
    Employee3.display();  
  
    return 0;  
}
```

★ **Output:**

A screenshot of a Windows command prompt window. The title bar shows the file path 'E:\C Programming\C++\Empl' and standard window controls. The output text is as follows:

```
Original (Employee1): Employee ID: 9, Name: Joshan  
Deep Copy (Employee2): Employee ID: 9, Name: Joshan  
Shallow Copy (Employee3): Employee ID: 9, Name: Joshan  
  
-----  
Process exited after 2.652 seconds with return value 0  
Press any key to continue . . .
```

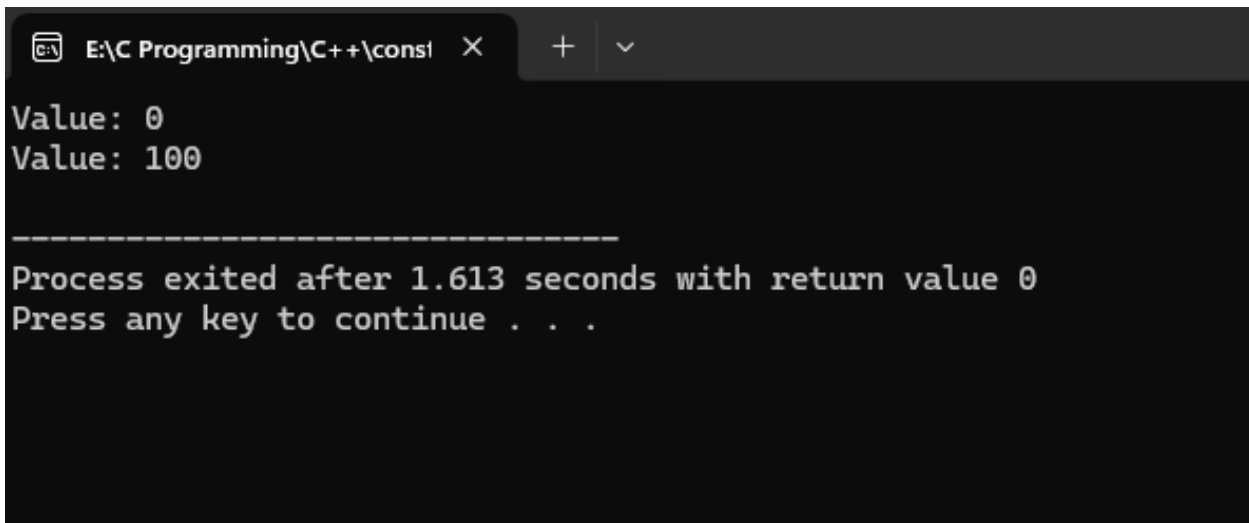

27). WAP showing Constructor overloading with appropriate example

```
#include <iostream>
using namespace std;

class Number {
    private:
        int value;
    public:
        Number() {
            value = 0;
        }
        Number(int x) {
            value = x;
        }
        void display() {
            cout << "Value: " << value << endl;
        }
};

int main() {
    Number n1;
    Number n2(100);
    n1.display();
    n2.display();
    return 0;
}
```

★ **Output:**

A screenshot of a terminal window showing the output of a C++ program. The window title is "E:\C Programming\C++\constl X". The output displays "Value: 0" and "Value: 100" on separate lines. Below this, a horizontal line of dashes is shown, followed by the text "Process exited after 1.613 seconds with return value 0" and "Press any key to continue . . .".

```
E:\C Programming\C++\constl X
Value: 0
Value: 100

-----
Process exited after 1.613 seconds with return value 0
Press any key to continue . . .
```

28).

// WAP showing concept of Destructor

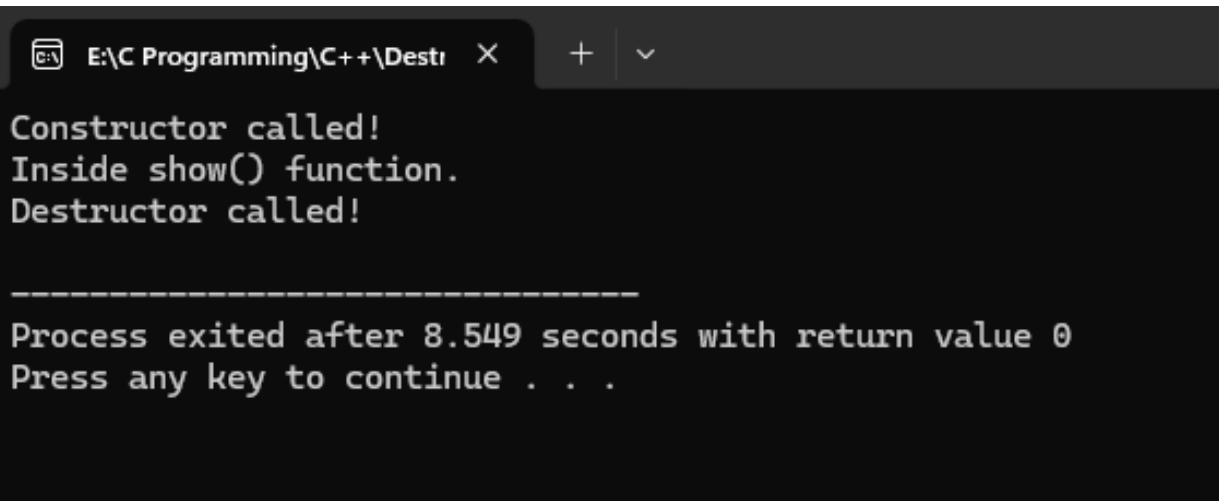
```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() {
        cout << "Constructor called!" << endl;
    }

    ~Demo() {
        cout << "Destructor called!" << endl;
    }

    void show() {
        cout << "Inside show() function." << endl;
    }
};

int main() {
    Demo d1;
    d1.show();
    return 0;
}
```

★ **Output:**A screenshot of a terminal window with a dark background. The window title bar shows the file path 'E:\C Programming\C++\Destri' and standard window controls. The output text is as follows:

```
Constructor called!
Inside show() function.
Destructor called!

-----
Process exited after 8.549 seconds with return value 0
Press any key to continue . . .
```

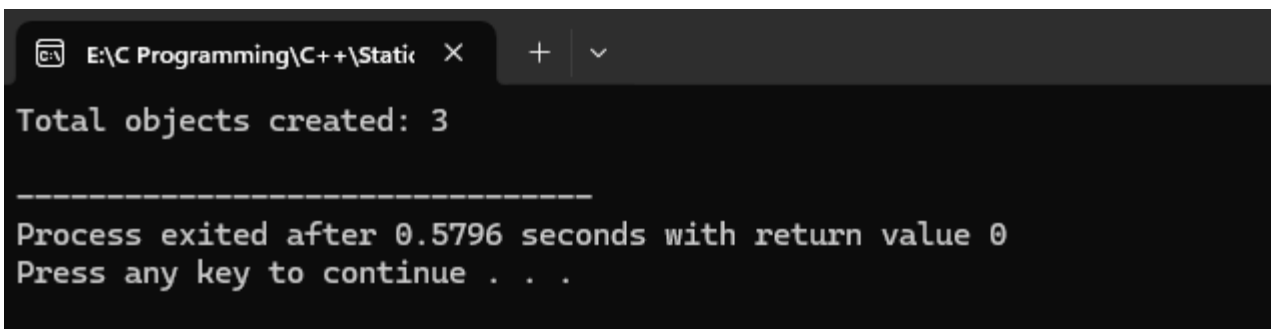
- 29). WAP to implement static data member and static data function. Each time object is instantiated, the constructor increases the static count. A static function getCount() is used to access the count. Call static function without object and dot operator.

```
#include <iostream>
using namespace std;

class Sample {
private:
    static int count;
public:
    Sample() {
        count++;
    }
    static void getCount() {
        cout << "Total objects created: " << count << endl;
    }
};

int Sample::count = 0;
int main() {
    Sample s1;
    Sample s2;
    Sample s3;
    Sample::getCount();
    return 0;
}
```

★ Output:



```
E:\C Programming\C++\Static  X  +  v
Total objects created: 3

-----
Process exited after 0.5796 seconds with return value 0
Press any key to continue . . .
```

- 30). The interest rate is static member common to all bank accounts. It can be changed via static function. Each object can calculate its own interest using shared interest rate.

```
#include <iostream>
using namespace std;

class Interest {
    private:
        double balance;
        int time;
        static float rate;

    public:
        void input() {
            cout << "Enter balance: ";
            cin >> balance;
            cout << "Enter time (in years): ";
            cin >> time;
        }
        void calculate() {
            double interest = (balance * rate * time) / 100;
            cout << "Balance: " << balance << endl;
            cout << "Time: " << time << " years" << endl;
            cout << "Interest: " << interest << endl;
        }

        static void setRate(float r) {
            rate = r;
        }
};

float Interest::rate = 5.0;

int main() {
    char change;
    float newRate;

    cout << "Default interest rate is 5%\n";
    cout << "Do you want to change the interest rate? (y/n): ";
    cin >> change;

    if (change == 'y' || change == 'Y') {
        cout << "Enter new interest rate: ";
        cin >> newRate;
        Interest::setRate(newRate);
    }
}
```

```
}

Interest i1;

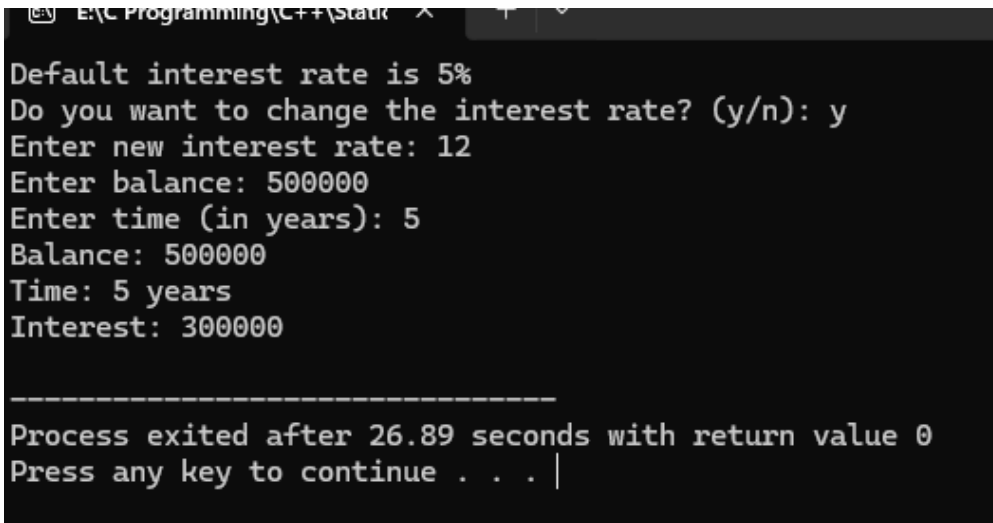
i1.input();

i1.calculate();

return 0;

}
```

★ Output:1

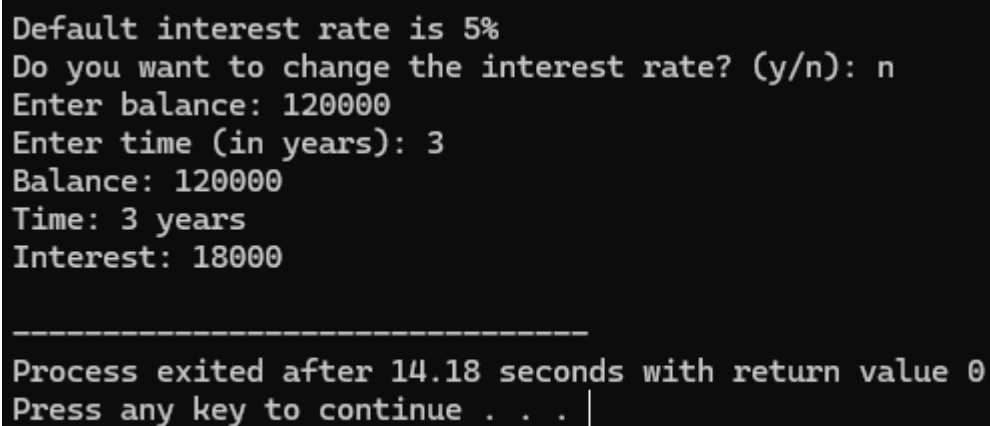


```
E:\C Programming\C++\Statu X 4 0

Default interest rate is 5%
Do you want to change the interest rate? (y/n): y
Enter new interest rate: 12
Enter balance: 500000
Enter time (in years): 5
Balance: 500000
Time: 5 years
Interest: 300000

-----
Process exited after 26.89 seconds with return value 0
Press any key to continue . . . |
```

★ Output:2



```
Default interest rate is 5%
Do you want to change the interest rate? (y/n): n
Enter balance: 120000
Enter time (in years): 3
Balance: 120000
Time: 3 years
Interest: 18000

-----
Process exited after 14.18 seconds with return value 0
Press any key to continue . . . |
```

31). // WAP to demonstrate passing an object to function by
 //a). Value
 //b). Reference
 //c). Pointer

```
#include <iostream>
using namespace std;

class Sample {
    private:
        int data;
    public:
        Sample(int d) {
            data = d;
        }
        void show() {
            cout << "Data = " << data << endl;
        }
        void setData(int d) {
            data = d;
        }
        int getData() {
            return data;
        }
};

// Pass by Value
void passByValue(Sample obj) {
    obj.setData(100);
    cout << "Inside passByValue(): ";
    obj.show();
}

// Pass by Reference
void passByReference(Sample &obj) {
    obj.setData(200);
    cout << "Inside passByReference(): ";
    obj.show();
}

// Pass by Pointer
void passByPointer(Sample *obj) {
    obj->setData(300);
    cout << "Inside passByPointer(): ";
    obj->show();
}
```

```

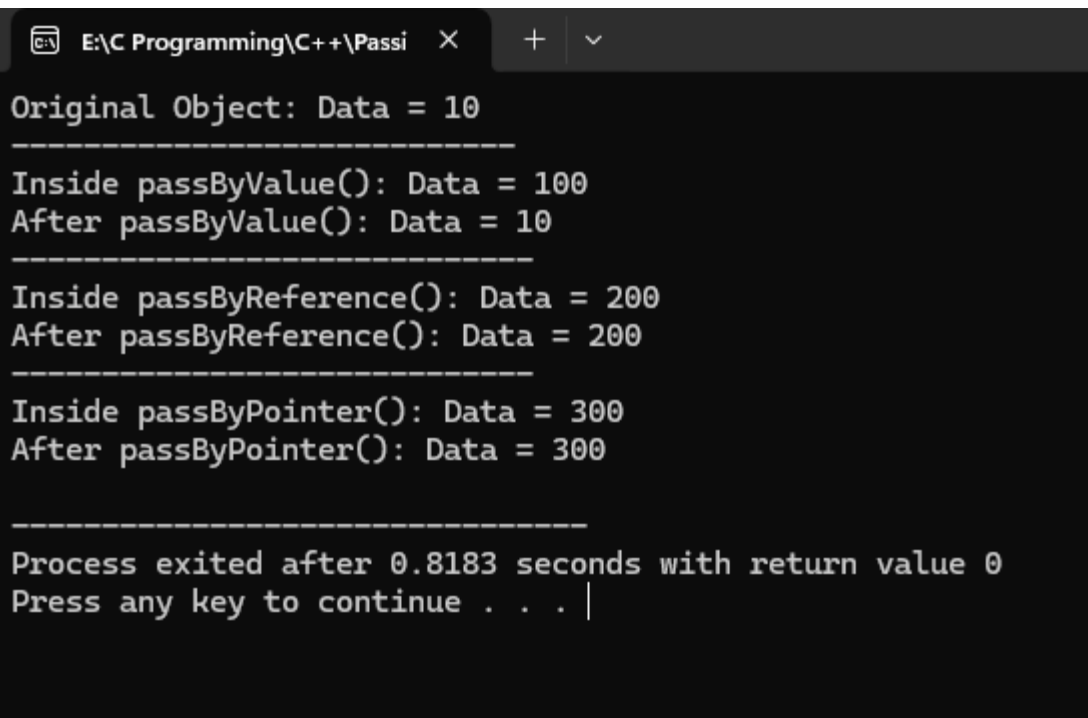
}

int main() {
    Sample s(10);

    cout << "Original Object: ";
    s.show();
    cout << "-----" << endl;
    passByValue(s);
    cout << "After passByValue(): ";
    s.show();
    cout << "-----" << endl;
    passByReference(s);
    cout << "After passByReference(): ";
    s.show();
    cout << "-----" << endl;
    passByPointer(&s);
    cout << "After passByPointer(): ";
    s.show();
    return 0;
}

```

★ **Output:**



```

Original Object: Data = 10
-----
Inside passByValue(): Data = 100
After passByValue(): Data = 10
-----
Inside passByReference(): Data = 200
After passByReference(): Data = 200
-----
Inside passByPointer(): Data = 300
After passByPointer(): Data = 300
-----
Process exited after 0.8183 seconds with return value 0
Press any key to continue . . . |

```

32). // WAP demonstrate returning an object to function by
 //a). Value
 //b). Reference
 //c). Pointer

```
#include <iostream>
using namespace std;

class Sample {
    private:
        int data;
    public:
        Sample(int d = 0) {
            data = d;
        }

        void setData(int d) {
            data = d;
        }

        void show() {
            cout << "Data = " << data << endl;
            cout << "-----" << endl;
        }

        int getData() {
            return data;
        }
};

// a) Return by Value
Sample returnByValue() {
    Sample obj(100);
    return obj;
}

// b) Return by Reference
Sample& returnByReference() {
    static Sample obj(200);
    return obj;
}

// c) Return by Pointer
Sample* returnByPointer() {
    static Sample obj(300);
    return &obj;
}
```



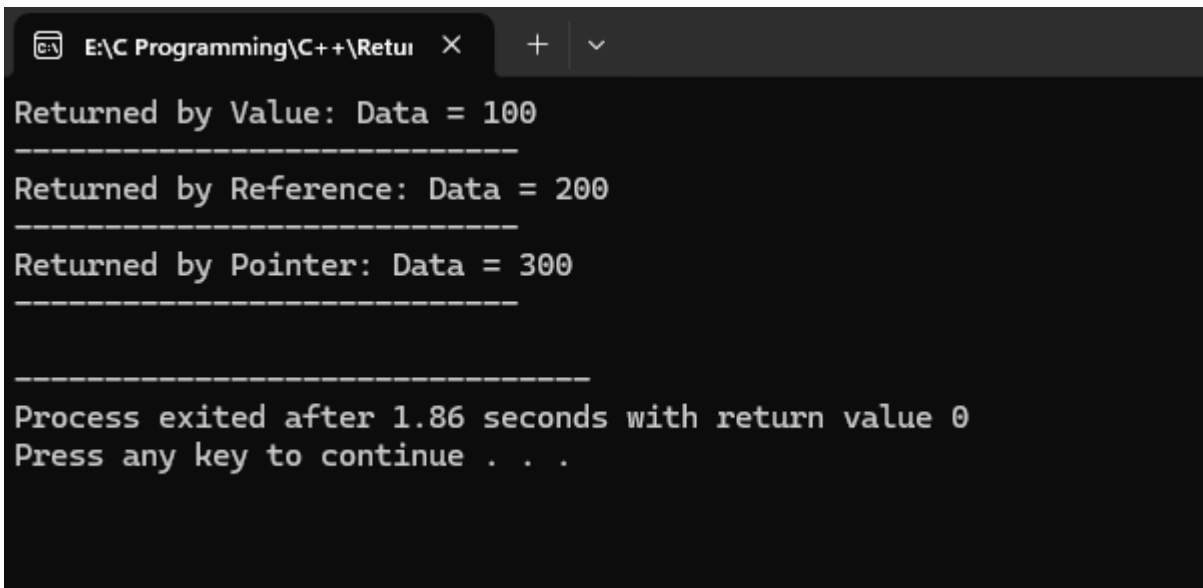
```
}

int main() {
    Sample a = returnByValue();
    cout << "Returned by Value: ";
    a.show();

    Sample& b = returnByReference();
    cout << "Returned by Reference: ";
    b.show();

    Sample* c = returnByPointer();
    cout << "Returned by Pointer: ";
    c->show();
    return 0;
}
```

★ **Output:**

A screenshot of a Windows command prompt window showing the output of a C++ program. The window title is "E:\C Programming\C++\Retur". The output text is as follows:

```
Returned by Value: Data = 100
-----
Returned by Reference: Data = 200
-----
Returned by Pointer: Data = 300
-----

-----
Process exited after 1.86 seconds with return value 0
Press any key to continue . . .
```

33). // WAP to design a class complex to represent complex numbers. Write a function sum(friend) to add two complex numbers. The function should return an object of type complex representing sum of two complex numbers.

```
#include<iostream>
using namespace std;

class complex
{
    int real, image;
public:
    friend complex sum(complex a,complex b);
    void show()
    {
        cout<<real<<"+"<<image<<"i"<<endl;
    }
    void get()
    {
        cout<<"Enter the real number:";
        cin>>real;
        cout<<"Enter the imaginary number:";
        cin>>image;
    }
};

complex sum (complex a,complex b)
{
    complex temp;
    temp.real=a.real+b.real;
    temp.image=a.image+b.image;
    return temp;
}

int main() {

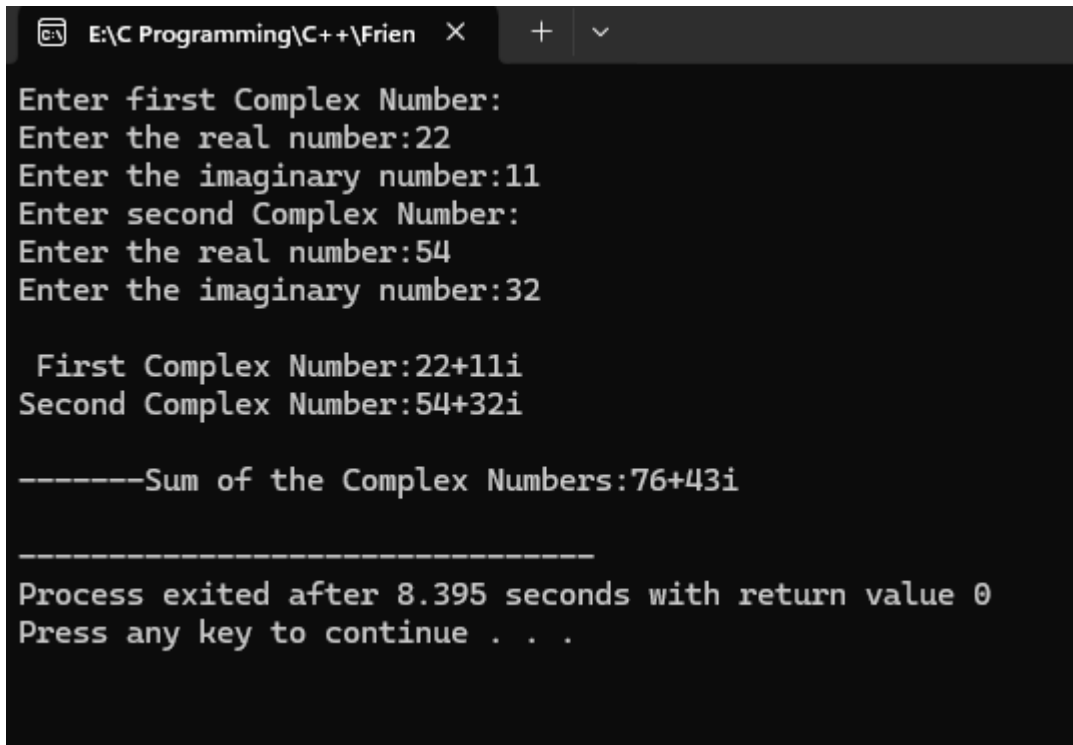
    complex c1;
    complex c2;
    cout << "Enter first Complex Number: "<<endl;
    c1.get();
    cout << "Enter second Complex Number: "<<endl;
    c2.get();
    cout << "\n First Complex Number:";
    c1.show();
    cout << "Second Complex Number:";
    c2.show();
}
```

```
complex result = sum(c1, c2);  
cout << "\n-----Sum of the Complex Numbers:";  
result.show();
```

```
return 0;
```

```
}
```

★ Output: -

A screenshot of a Windows command prompt window with a dark background. The window title bar shows the file path "E:\C Programming\C++\Frien" and standard window controls. The output text is as follows:

```
Enter first Complex Number:  
Enter the real number:22  
Enter the imaginary number:11  
Enter second Complex Number:  
Enter the real number:54  
Enter the imaginary number:32  
  
First Complex Number:22+11i  
Second Complex Number:54+32i  
  
-----Sum of the Complex Numbers:76+43i  
  
-----  
Process exited after 8.395 seconds with return value 0  
Press any key to continue . . .
```

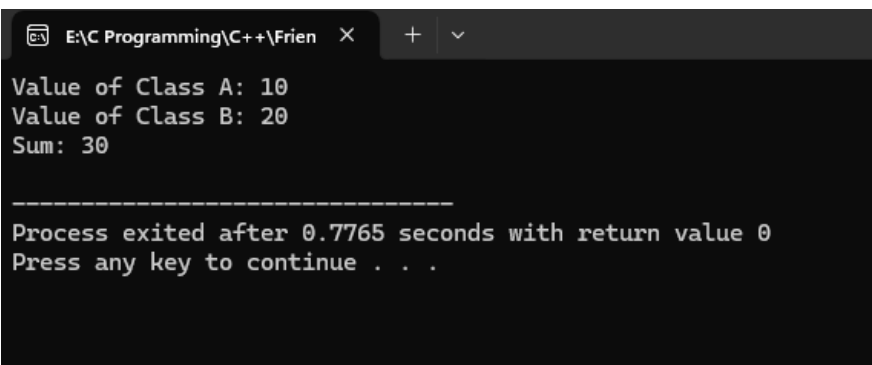
34). // WAP showing friend function as bridge between two class.

```
#include <iostream>
using namespace std;

class B;
class A {
    private:
        int a;
    public:
        A (int val) {
            a = val;
        }
        friend void showSum(A, B);
};
class B {
    private:
        int b;
    public:
        B (int val) {
            b = val;
        }
        friend void showSum(A, B);
};
void showSum(A objA, B objB) {
    cout << "Value of Class A: " << objA.a << endl;
    cout << "Value of Class B: " << objB.b << endl;
    cout << "Sum: " << objA.a + objB.b << endl;
}
int main() {

    A objA(10);
    B objB(20);
    showSum(objA, objB);
    return 0;
}
```

★ Output: -



```
E:\C Programming\C++\Frien X + v
Value of Class A: 10
Value of Class B: 20
Sum: 30

-----
Process exited after 0.7765 seconds with return value 0
Press any key to continue . . .
```